

Rationale

The goals I set for myself in this project were as follows: Firstly, I had to design and implement a relational database system adequate to store, manage, and analyse cooking recipes. Since the beginning, this database has aimed at normalisation on the one side, clarity, and scalability on the other. Therefore, the first thing to know is that I tried to establish a clear schema, staying close to the principles of relational database design while at the same time providing sufficiently fast querying and usability for data retrieval. Another aim was to show the most important area entities. Those I found to be Users, Recipes, Categories, Ingredients;

These tables were built with the relevant primary keys, and foreign keys were used to ensure referential integrity. As an example, the Recipes table has connections to both users and categories and ratings to users and recipes. I used foreign keys in this way to guarantee that the information stays correct for several correlated tables:

I applied normalisation principles up to the third normal form, thus avoiding data redundancy and ensuring that each table serves a single purpose. In this regard, the join table Recipe_Ingredients is especially relevant. It eliminates any restrictions on the structuring of ingredients and their number, effectively storing all recipe data. In addition, the data types in the database have been selected carefully. For example, the ratings table has a check constraint restricting the data value between 1 and 5 to ensure its validity.

Indexing and optimisation: I ensured that primary keys were indexed automatically and used the show index command to confirm it.

SQL queries: The project shows a complete list of operations related to insertion, updating, deletion, filtering, joining, and subqueries. I prepared it with simplicity and rapidity in mind. I even managed to create a join query able to merge three tables (Recipes, Recipe_Ingredients, and Ingredients) to provide the user with a view of ingredients for each recipe.

This query is responsible for the calculation of the highest-rated recipe by using AVG and GROUP BY. It then gets the corresponding rating and utilises order by and limit to return only the top result. On the other hand, the subquery determines the user who rated the above-defined highest recipe. This element demonstrates nested logic as the possibility of adequately integrating queries. As it concerns adherence to best practices, throughout the project, I have written all queries in a way that the aliases were used to make them more readable. Also, logical clauses were placed strategically to make the

file more accessible. The operations are also confirmed by screenshots taken from MySQL Workbench.

Looking back at the general implementation, I think the database is structured and normalised based on logic and requirements. If I had had more time, in subsequent passes, I would also have considered more optimisation strategies, including proper profiling of the most frequent queries and index tuning to target these patterns. However, the existing solution meets all of the project specifications and reflects a basis for a solid comprehension of database design, SQL language fundamentals, and data manipulation.